📝 Java Interfaces Summary: The Contract

In Java, an **Interface** is a blueprint of a class that defines a **contract** for any class that chooses to implement it. It is a tool used to achieve full **abstraction** and to support the concept of **multiple inheritance of type**.

1. Key Characteristics

An interface is fundamentally different from a regular class:

Feature	Interface Rule	Notes
Methods	All methods are implicitly public and abstract.	The implementing class *must* provide a concrete body for these methods. (Since Java 8, default and static methods are also allowed).
Fields	All fields are implicitly public static final.	They act as constants and cannot be instance variables.
Instantiation	Cannot be instantiated directly.	You cannot use new InterfaceName() to create an object.
Keyword	Classes use the implements keyword to fulfill the contract.	A class can implement multiple interfaces.

2. Why Use Interfaces?

Interfaces solve two major design problems in Java:

1. **Defining a Contract:** They guarantee that any class implementing the interface will have certain methods, providing a reliable structure for larger systems.
2. **Achieving Multiple Inheritance:** Java does not allow a class to extend (inherit from) more than one class. However, a class **can implement multiple interfaces**, allowing it to inherit behavior specifications from many sources.

3. Syntax and Implementation

Step 1: Define the Interface

The interface declares *what* the implementing class must do, but not *how* it does it.

```
// Interface Definition
public interface Drivable {
    // Implicitly public static final
    int MAX_SPEED = 100;

    // Implicitly public abstract
    void startEngine();
    void stopEngine();
}
```

Step 2: Implement the Interface

A concrete class uses the implements keyword and **must** provide method bodies for all abstract methods defined in the interface.

```
// Class Implementation
public class RaceCar implements Drivable {
    // Implementation of the interface methods is mandatory
    @Override
    public void startEngine() {
        System.out.println("RaceCar engine roars to life!");
    }

    @Override
    public void stopEngine() {
        System.out.println("RaceCar slows down.");
    }

    // We can access the constant defined in the interface
    public void checkSpeedLimit() {
        System.out.println("Maximum allowed speed: " + MAX_SPEED);
    }
}
```

Step 3: Using the Interface as a Type

You can use the Interface name as the reference type. This is powerful because it allows you to swap out implementations easily.

```
public class TestDrive {
    public static void main(String[] args) {
        // We declare the reference as the Interface type (Drivable)
        Drivable myVehicle = new RaceCar();

        // We can only call methods defined in the Drivable interface
        myVehicle.startEngine();
        myVehicle.stopEngine();
    }
}
```